ARCHITECTURE BRIEF

Recommended technology stack for a multi-tenant inventory & POS SaaS

Aligned to the Inventory Management System UI/UX Mockup Screen List.
Opinionated MVP stack: one language, one application, real stock ledgers.
Prepared for product and engineering planning. Not a build-everything catalogue.

12 October 2026

Next.js · PostgreSQL · Paystack

1. Purpose of this brief

The mockup guide lists roughly 270 candidate screens and a much smaller MVP. This document recommends a single, seamless technology path so the MVP can ship without a rewrite when purchases, branches, billing, and reports arrive.

Seamless here means: one TypeScript codebase for marketing, tenant app, POS, and APIs; one transactional database for stock; Nigerian payments that confirm on the server; hosting that matches a SaaS rental model.

> **Guiding principle**
> Keep inventory and sales accurate before adding advanced modules. Tenant isolation and access control matter more than screen count. Do not build the full catalogue in v1.

2. Product constraints that drive the stack

- Multi-business SaaS: every product, sale, customer, and staff record belongs to one business.
- Stock must move only through auditable transactions (sale, receipt, adjustment, transfer).
- Roles: owner, manager, cashier — cashiers must not freely change cost or full P&L.
- Desktop dashboard and tablet POS; USB/Bluetooth scanners act like a keyboard.
- Annual subscription billed in Nigeria (Paystack) with webhook confirmation.
- Later: barcode labels, accounting, e-commerce, true offline POS — without replacing the core.

3. Recommended stack (use this path)

Stay in one Next.js application. Do not add a separate Nest, Laravel, or microservice API for version one.

Layer	Choice	Why it fits this product
Application	Next.js (App Router) + TypeScript	Landing, auth, dashboard, inventory, POS and admin in one repo and one deploy.
User interface	Tailwind CSS + shadcn/ui	Turns numbered mockup screens into consistent pages; works on tablet POS.
Database	PostgreSQL + Prisma	ACID stock updates, foreign keys, joins for reports, purchase orders and ledgers.
Authentication	Auth.js (credentials) + httpOnly cookies	Session holds businessId and role. Never trust IDs sent by the browser.
Passwords	bcrypt (or Argon2)	Hashed at rest; reset flow via email.
Files / images	Cloudinary or S3-compatible storage	Product images and receipts without bloating the database.
Payments	Paystack + server webhooks	Annual plans and later POS card/transfer; status confirmed by the server.
Hosting (app)	Vercel	Fits Next.js; preview deploys for each change.
Hosting (data)	Neon or Supabase Postgres	Managed Postgres with backups; no Mongo cluster to operate.
Email	Resend	Verify email, staff invite, password reset.
Background jobs	Inngest (add when needed)	CSV import, large reports, renewals, low-stock emails.

What not to use for MVP

- MongoDB — weak for “sale must decrement stock or the whole sale fails,” POs, and financial reports.

- Native iOS/Android first — the guide is a web POS. Ship a PWA. Offline is phase two.
- A second backend framework — two deploys, two auth stories, slower POS.
- Microservices, Kafka, Kubernetes — one database and one app until tenant volume demands it.
- Building all 270+ screens — the guide itself says the list is a planning catalogue.

4. Why PostgreSQL instead of MongoDB

An earlier inventory POS note suggested MongoDB Atlas. For this mockup’s stock, purchases, partial goods receipts, and reports, relational data is the seamless choice.

- A sale, a stock decrement, and a ledger row must commit together or not at all.
- business_id foreign keys plus row-level checks make tenant isolation enforceable.
- Reports (sales, valuation, low stock, staff performance) are ordinary SQL, not map-reduce.
- Purchase orders, partial receipts, and variances are relational documents with line items.

Mongo remains acceptable only if you already operate it and you still wrap every stock change in a multi-document transaction. Prefer Postgres for a greenfield build.

5. Core data model

Every tenant-owned row includes business_id taken from the authenticated session. Example query shape: find product where id = :id and business_id = current_user.business_id.

Minimum tables for the recommended MVP screens:

- businesses, users, roles / permissions
- products, categories, variants / attributes
- locations (shop floor, warehouse) — even if only one branch at first
- stock_ledger — who, what, quantity, before/after, reason, timestamp
- sales, sale_items (unit price frozen on the line so later price edits do not rewrite history)
- customers, suppliers, purchase_orders, purchase_order_lines, goods_receipts
- subscriptions, payments (Paystack reference, status, period)

> **Stock rule**
> Never update product.stock_qty in isolation. Write a stock_ledger row inside a database transaction; derive on-hand quantity from the ledger (or update on-hand in the same transaction). Sales decrease stock. Goods received increase it. Returns and adjustments follow the same ledger.

6. Mapping stack to MVP screens

Section 3 of the mockup guide is the only build target for v1. Everything else waits.

MVP group	Screens	Stack handling
Account & setup	Sign up, login, business profile, setup wizard	Auth.js + businesses table + Resend verification.
Dashboard & products	Main dashboard, product list/add/details, categories	Prisma queries scoped by session business_id.
Inventory control	Stock levels, adjustment, ledger, low stock, count	stock_ledger + transactional updates; no negative stock by default.
Sales / POS	POS, cart, payment method, confirmation, receipt, history	Tablet layout; scanner types into search; Paystack later for card.
Purchases & contacts	Suppliers, PO list, goods received, customers	Same Postgres schema; no new service.
Reports & admin	Sales report, inventory report, staff, roles, settings	SQL aggregates; cashier role cannot see cost or full P&L.
Billing	Plan, checkout, webhook, expired/restricted	Paystack; server activates or extends subscription.

Suggested design and build order

Dashboard → Product list → Add product → Stock adjustment → POS → Purchase receiving → Sales report. That sequence proves the core loop before variants, transfers, and custom reports.

7. Security and tenant isolation

- business_id always comes from the session, never from a form field the browser can edit.
- Hash passwords. HttpOnly, Secure, SameSite cookies. Short-lived sessions with refresh if needed.
- Authorize every mutation: cashiers POS-only; managers inventory; owners staff, billing, settings.
- Critical test: Business A must not read or write Business B’s product, sale, customer, or staff IDs via the API.
- Rate-limit login. Audit log for stock, permissions, and billing events.

8. POS, barcode, and devices

Treat the POS as a first-class web screen, not a native app in v1.

- Layout for landscape tablet; large tap targets; cart always visible.
- USB/Bluetooth scanners send keystrokes into the search field; also allow manual barcode entry.
- Hold sale, receipt print/PDF, cash / transfer / card as payment methods. Split payment can wait.
- True offline POS (IndexedDB + sync + conflict handling) is explicitly out of MVP. If the network drops, show a clear connection-lost state.

9. Payments and subscriptions

Businesses choose an annual plan. Paystack collects payment. A webhook (charge.success) is the source of truth: the server sets subscription start, expiry, and status. Do not activate a plan from a browser callback alone.

When a subscription expires, restrict the tenant (read-only or blocked POS) according to product rules. Platform admins see businesses, plans, and payments in a separate admin area.

10. Hosting and operations

- Vercel for the Next.js app. Neon or Supabase for Postgres. Environment secrets for AUTH_SECRET and Paystack keys.

- Vercel is enough until reports or imports run long. Then add Inngest (or Trigger.dev) for background work.
- Daily database backups. Do not store live secrets in the repo.
- Source control on GitHub. Preview deployments for pull requests.

11. What to add later (without changing the spine)

From the long screen list	Add later
Branches, warehouses, transfers	locations table + transfer documents on the same Postgres.
Register open/close, split tender	Extra sale and till fields in Next.js.
CSV import, custom report builder	Inngest jobs + queued exports.
Accounting / Shopify / delivery	Outbound webhooks; do not embed in v1.
Offline POS	IndexedDB queue, sync, conflict rules after online POS is trusted.
Mobile application	Reuse the API routes; do not fork business logic.

12. Explicitly out of MVP

- True offline POS, complex accounting, payroll, advanced CRM.
- Large-scale analytics, automatic delivery integrations.
- Multi-country multi-currency as a first-class system.
- Enterprise permission engines beyond owner / manager / cashier.

13. Decision summary

> **Adopt this stack**
> Next.js + TypeScript + Tailwind / shadcn/ui + PostgreSQL / Prisma + Auth.js + Paystack + Vercel + Neon (or Supabase) + Resend. Add Inngest when jobs appear. Add IndexedDB only after the online POS is accurate.

This is the least-friction path for a Tracepos-style multi-tenant inventory SaaS: one language, one application, real stock transactions, Nigerian billing, and room for the rest of the mockup list when the core workflow is proven.

Next engineering step: Prisma schema and folder map limited to section-3 screens only.